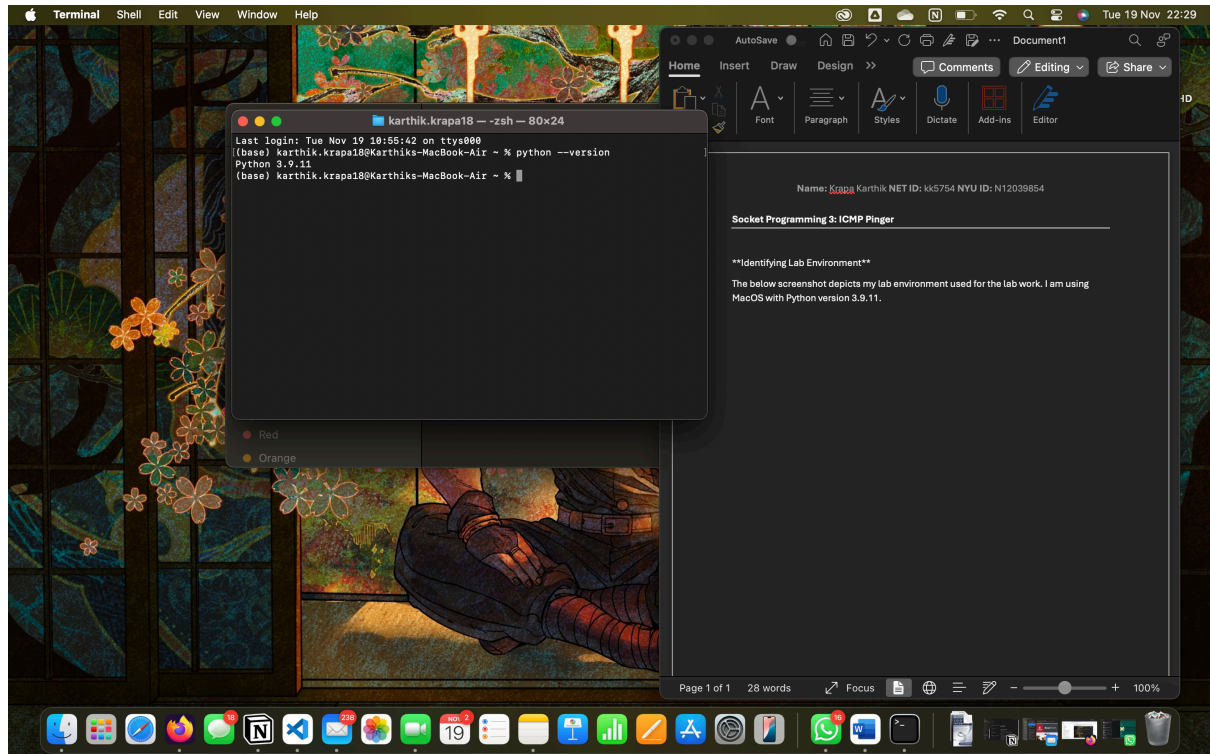


Socket Programming 3: ICMP Pinger

Identifying Lab Environment

The below screenshot depicts my lab environment used for the lab work. I am using MacOS with Python version 3.9.11.



1. **ICMP Pinger Lab:** Client Code Completed with commented lines for explanation of code.

client.py

```
2. from socket import *
3. import os
4. import sys
5. import struct
6. import time
7. import select
8.
9. ICMP_ECHO_REQUEST = 8 # ICMP Type
10. IP_HEADER_SIZE = 20 #Standard size
11.
12. def checksum(string):
13.     csum = 0 #initialize checksum
14.     countTo = (len(string) // 2) * 2
15.     count = 0
16.     while count < countTo:
17.         thisVal = (string[count + 1]) * 256 +
18.         (string[count])
19.         csum += thisVal
20.         csum &= 0xffffffff #Keeping checksum within 32 bytes
21.         count += 2
22.     if countTo < len(string):
23.         csum += (string[len(string) - 1])
24.         csum &= 0xffffffff
25.     csum = (csum >> 16) + (csum & 0xffff)
26.     csum = csum + (csum >> 16)
27.     answer = ~csum
28.     answer = answer & 0xffff
29.     answer = answer >> 8 | (answer << 8 & 0xff00)
30.     return answer
31.
32. def receiveOnePing(mySocket, ID, timeout, destAddr):
33.     timeLeft = timeout #setting timeout value
34.     while True:
35.         startedSelect = time.time() #start time
36.         whatReady = select.select([mySocket], [], [],
37.         timeLeft) #incoming data
38.         howLongInSelect = (time.time() - startedSelect)
39.         #time spent
40.         if whatReady[0]==[]: # Timeout
41.             return (None, None)
```

```
39.         timeReceived = time.time() #packet received time
40.         recPacket, addr = mySocket.recvfrom(1024) #receive
           packet and sender's address
41.
42.         #Extract ICMP header fields from the received packet
43.         #fill in start
44.         icmpHeader = recPacket[IP_HEADER_SIZE:IP_HEADER_SIZE
           + 8]
45.         icmpType, icmpCode, checksum, packetID, sequence =
           struct.unpack("bbHHh", icmpHeader)
46.         if packetID == ID:
47.             #Extract the data payload
48.             data = recPacket[IP_HEADER_SIZE + 8:]
49.             sendTimestamp = struct.unpack("d", data)[0]
50.             return ((timeReceived - sendTimestamp) * 1000,
           #RTT in ms
           (icmpType, icmpCode, checksum, packetID,
           sequence, sendTimestamp))
51.         #fill in end
52.         timeLeft -= howLongInSelect
53.         if timeLeft <= 0:
54.             return (None, None)
55.
56.
57.     def sendOnePing(mySocket, destAddr, ID):
58.         # Header is type (8), code (8), checksum (16), id (16),
           sequence (16)
59.         myChecksum = 0
60.         # Create dummy header with 0 checksum
61.         header = struct.pack("bbHHh", ICMP_ECHO_REQUEST, 0,
           myChecksum, ID, 1)
62.         data = struct.pack("d", time.time())
63.         # Calculate checksum on header + data
64.         myChecksum = checksum(header + data)
65.         # Put checksum into the header
66.         if sys.platform == 'darwin':
67.             myChecksum = htons(myChecksum) & 0xffff # Convert
           for MacOS
68.         else:
69.             myChecksum = htons(myChecksum)
70.             header = struct.pack("bbHHh", ICMP_ECHO_REQUEST, 0,
           myChecksum, ID, 1)
71.             packet = header + data
72.             mySocket.sendto(packet, (destAddr, 1)) # Destination
           must be tuple
```

```
73.         return time.time() # Return send time to calculate RTT
       later
74.
75.     def doOnePing(destAddr, timeout):
76.         icmp = getprotobyname("icmp")
77.         mySocket = socket(AF_INET, SOCK_RAW, icmp)
78.         myID = os.getpid() & 0xFFFF # Get process ID for unique
       identifier
79.         sendOnePing(mySocket, destAddr, myID)
80.         result = receiveOnePing(mySocket, myID, timeout,
       destAddr)
81.         mySocket.close()
82.         return result
83.
84.     def ping(host, timeout=1):
85.         dest = gethostbyname(host)
86.         resps = []
87.         print("Pinging " + dest + " using Python:")
88.         print("")
89.         for i in range(5): # Send 5 pings
90.             result = doOnePing(dest, timeout)
91.             resps.append(result)
92.             time.sleep(1)
93.         print(resps)
94.         return resps
95.
96.     if __name__ == '__main__':
97.         ping("google.com")
98.
```

checksum: Computes a checksum for error checking.

sendOnePing: Constructs and sends an ICMP Echo Request.

receiveOnePing: Receives an ICMP Echo Reply and extracts details.

doOnePing: Combines sending and receiving logic for a single ping.

ping: Sends multiple pings and displays the results.

Extra Credit 1

Extra-1.py

```
from socket import *
import os
```

```
import sys
import struct
import time
import select

ICMP_ECHO_REQUEST = 8 # ICMP Type
IP_HEADER_SIZE = 20

def checksum(data):
    csum = 0
    countTo = (len(data) // 2) * 2
    count = 0
    while count < countTo:
        thisVal = (data[count + 1]) * 256 + (data[count])
        csum += thisVal
        csum &= 0xffffffff
        count += 2
    if countTo < len(data):
        csum += (data[len(data) - 1])
        csum &= 0xffffffff
    csum = (csum >> 16) + (csum & 0xffff)
    csum = csum + (csum >> 16)
    answer = ~csum
    answer = answer & 0xffff
    answer = answer >> 8 | (answer << 8 & 0xff00)
    return answer

def receive_one_ping(mySocket, ID, timeout, dest_addr):
    timeLeft = timeout
    while True:
        startedSelect = time.time()
        whatReady = select.select([mySocket], [], [], timeLeft)
        howLongInSelect = (time.time() - startedSelect)
        if whatReady[0]==[]: # Timeout
            return (None, None)
        timeReceived = time.time()
        recPacket, addr = mySocket.recvfrom(1024)

        # Extract ICMP header fields from the received packet
        icmpHeader = recPacket[IP_HEADER_SIZE:IP_HEADER_SIZE + 8]
        icmpType, icmpCode, checksum, packetID, sequence =
struct.unpack("bbHHh", icmpHeader)
        if packetID == ID:
            # Extract the data payload
```

```
        data = recPacket[IP_HEADER_SIZE + 8:]
        sendTimestamp = struct.unpack("d", data)[0]
        return ((timeReceived - sendTimestamp) * 1000, # RTT in ms
                (icmpType, icmpCode, checksum, packetID, sequence,
sendTimestamp))

    timeLeft -= howLongInSelect
    if timeLeft <= 0:
        return (None, None)

def send_one_ping(sock, dest_addr, ID):
    checksum_val = 0
    header = struct.pack("bbHHh", ICMP_ECHO_REQUEST, 0, checksum_val,
ID, 1)
    data = struct.pack("d", time.time())
    checksum_val = checksum(header + data)
    checksum_val = htons(checksum_val)
    header = struct.pack("bbHHh", ICMP_ECHO_REQUEST, 0, checksum_val,
ID, 1)
    packet = header + data
    sock.sendto(packet, (dest_addr, 1))
    return time.time()

def do_one_ping(dest_addr, timeout):
    icmp = getprotobyname("icmp")
    sock = socket(AF_INET, SOCK_RAW, icmp)
    ID = os.getpid() & 0xFFFF
    send_one_ping(sock, dest_addr, ID)
    result = receive_one_ping(sock, ID, timeout, dest_addr)
    sock.close()
    return result

def ping(host, timeout=1):
    dest = gethostbyname(host)
    print(f"Pinging {dest} using Python:")
    min_rtt, max_rtt, total_rtt, total_received = float('inf'), 0, 0, 0

    for i in range(5):
        rtt, _ = do_one_ping(dest, timeout)
        if rtt:
            min_rtt = min(min_rtt, rtt)
            max_rtt = max(max_rtt, rtt)
            total_rtt+=rtt
            total_received += 1
```

```
        time.sleep(1)

    packet_loss = (5 - total_received) / 5 * 100
    print("\n--- PING STATISTICS ---")
    print(f"Packets Sent: 5, Packets Received: {total_received}, Packet
Loss: {packet_loss:.1f}%")
    if total_received > 0:
        print(f"RTT (ms): min={min_rtt:.2f}ms,
avg={({total_rtt/total_received}):.2f}ms, max={max_rtt:.2f}ms")

if __name__ == "__main__":
    ping("google.com")
```

Extra Credit 2:

Extra-2.py

```
from socket import *
import os
import sys
import struct
import time
import select

ICMP_ECHO_REQUEST = 8 # ICMP Type
IP_HEADER_SIZE = 20

ICMP_ERROR_TYPES = {
    3: "Destination Unreachable",
    5: "Redirect",
    11: "Time Exceeded",
    12: "Parameter Problem",
    4: "Source Quench",
}

ICMP_ERROR_CODES = {
    3: {
        0: "Net Unreachable",
        1: "Host Unreachable",
        2: "Protocol Unreachable",
        3: "Port Unreachable",
        4: "Fragmentation Needed and Don't Fragment was Set",
        5: "Source Route Failed",
```

```
        6: "Destination Network Unknown",
        7: "Destination Host Unknown",
        8: "Source Host Isolated",
        9: "Communication with Destination Network is Administratively
Prohibited",
        10: "Communication with Destination Host is Administratively
Prohibited",
        11: "Destination Network Unreachable for Type of Service",
        12: "Destination Host Unreachable for Type of Service",
        13: "Communication Administratively Prohibited",
        14: "Host Precedence Violation",
        15: "Precedence cutoff in effect",
    },
    4: {0: "No Code"},
    5: {
        0: "Redirect Datagram for the Network (or subnet)",
        1: "Redirect Datagram for the Host",
        2: "Redirect Datagram for the Type of Service and Network",
        3: "Redirect Datagram for the Type of Service and Host",
    },
    11: {0: "Time to Live exceeded in Transit", 1: "Fragment Reassembly
Time Exceeded"},
    12: {0: "Pointer indicates the error", 1: "Missing a Required
Option", 2: "Bad Length"},
}

def parse_icmp_error(icmp_type, icmp_code):
    error_type = ICMP_ERROR_TYPES.get(icmp_type, None)
    if error_type is None:
        return "no error"
    error_msg = ICMP_ERROR_CODES.get(icmp_type, {}).get(icmp_code,
None)
    if error_msg is None:
        return "invalid error code"
    return f"error type: {error_type} | error message: {error_msg}"

def checksum(data):
    csum = 0
    countTo = (len(data) // 2) * 2
    count = 0
    while count < countTo:
        thisVal = (data[count + 1]) * 256 + (data[count])
        csum += thisVal
        csum &= 0xffffffff
```

```
        count += 2
    if countTo < len(data):
        csum += (data[len(data) - 1])
        csum &= 0xffffffff
    csum = (csum >> 16) + (csum & 0xffff)
    csum = csum + (csum >> 16)
    answer = ~csum
    answer = answer & 0xffff
    answer = answer >> 8 | (answer << 8 & 0xff00)
    return answer

def receive_one_ping(mySocket, ID, timeout, dest_addr):
    timeLeft = timeout
    while True:
        startedSelect = time.time()
        whatReady = select.select([mySocket], [], [], timeLeft)
        howLongInSelect = (time.time() - startedSelect)
        if whatReady[0]==[]: # Timeout
            return (None, None)
        timeReceived = time.time()
        recPacket, addr = mySocket.recvfrom(1024)

        # Extract ICMP header fields from the received packet
        icmpHeader = recPacket[IP_HEADER_SIZE:IP_HEADER_SIZE + 8]
        icmpType, icmpCode, checksum, packetID, sequence =
struct.unpack("bbHHh", icmpHeader)
        parse_result = parse_icmp_error(icmpType, icmpCode)
        print(f"Parsing icmp type and code: {parse_result}")
        if packetID == ID:
            # Extract the data payload
            data = recPacket[IP_HEADER_SIZE + 8:]
            sendTimestamp = struct.unpack("d", data)[0]
            return ((timeReceived - sendTimestamp) * 1000, # RTT in ms
                    (icmpType, icmpCode, checksum, packetID, sequence,
sendTimestamp))

        timeLeft -= howLongInSelect
        if timeLeft <= 0:
            return (None, None)

def send_one_ping(sock, dest_addr, ID):
    checksum_val = 0
    header = struct.pack("bbHHh", ICMP_ECHO_REQUEST, 0, checksum_val,
ID, 1)
```

```
data = struct.pack("d", time.time())
checksum_val = checksum(header + data)
checksum_val = htons(checksum_val)
header = struct.pack("bbHHh", ICMP_ECHO_REQUEST, 0, checksum_val,
ID, 1)
packet = header + data
sock.sendto(packet, (dest_addr, 1))
return time.time()

def do_one_ping(dest_addr, timeout):
    icmp = getprotobyname("icmp")
    sock = socket(AF_INET, SOCK_RAW, icmp)
    ID = os.getpid() & 0xFFFF
    send_one_ping(sock, dest_addr, ID)
    result = receive_one_ping(sock, ID, timeout, dest_addr)
    sock.close()
    return result

def ping(host, timeout=1):
    dest = gethostbyname(host)
    print(f"Pinging {dest} using Python:")
    min_rtt, max_rtt, total_rtt, total_received = float('inf'), 0, 0, 0

    for i in range(5):
        rtt, _ = do_one_ping(dest, timeout)
        if rtt:
            min_rtt = min(min_rtt, rtt)
            max_rtt = max(max_rtt, rtt)
            total_rtt+=rtt
            total_received += 1
        time.sleep(1)

    packet_loss = (5 - total_received) / 5 * 100
    print("\n--- PING STATISTICS ---")
    print(f"Packets Sent: 5, Packets Received: {total_received}, Packet
Loss: {packet_loss:.1f}%")
    if total_received > 0:
        print(f"RTT (ms): min={min_rtt:.2f}ms,
avg={({total_rtt/total_received}):.2f}ms, max={max_rtt:.2f}ms")

if __name__ == "__main__":
    ping("google.com")
```